# Risk-Aware Deep Reinforcement Learning for Daily Portfolio Allocation

***Dissecting Temporal Representation and Online Reward Mechanics. A Controlled Ablation of Temporal Memory, Reward Shaping, and Turnover Regularization***

## 1. Project Overview

Financial markets are characterized by low signal-to-noise ratios, high volatility, and non-stationary dynamics. Deep Reinforcement Learning (DRL) is theoretically well-suited for sequential trading decisions because it optimizes for cumulative future rewards rather than immediate prediction accuracy. However, standard DRL algorithms often fail in real-world deployment due to their inability to capture time-series momentum, their blindness to drawdown risk, and their tendency to generate high-turnover policies that succumb to transaction frictions. This BTP-1 project conducts a controlled empirical investigation to systematically evaluate the architectural and reward-design mechanisms required to stabilize a Proximal Policy Optimization (PPO) agent for daily equity trading.

---

## 2. Literatures

- **Zou et al. (2023)** link
  - Full Title: A Novel Deep Reinforcement Learning Based Automated Stock Trading System Using Cascaded LSTM Networks
  - Authors: Jie Zou, Jiashu Lou, Baohua Wang, Sixue Liu
  - Journal: Expert Systems With Applications
- **Huang et al. (2024)** link
  - Full Title: A Self-Rewarding Mechanism in Deep Reinforcement Learning for Trading Strategy Optimization
  - Authors: Yuling Huang, Chujin Zhou, Lin Zhang, Xiaoping Lu
  - Journal: Mathematics
- **Liu et al. / FinRL-Meta (2024)** link
  - Full Title: Dynamic datasets and market environments for financial reinforcement learning
  - Authors: Xiao-Yang Liu, Ziyi Xia, Hongyang Yang, Jiechao Gao, Daochen Zha, Ming Zhu, Christina Dan Wang, Zhaoran Wang, Jian Guo
  - Journal: Machine Learning
- **Millea (2021)** link
  - Full Title: Deep Reinforcement Learning for Trading—A Critical Survey
  - Author: Adrian Millea
  - Journal: Data
- **Wang & Liu (2025)** link
  - Full Title: ART-DRL: Adaptive Risk-Sensitive Deep Reinforcement Learning

---

## 3. Research Question

**Main Research Question:** How do temporal memory, dense risk-adjusted reward shaping, and turnover regularization independently and cumulatively affect the risk-adjusted out-of-sample performance of a **PPO-based** daily trading agent?

This translates into three specific, testable hypotheses: * **H1:** LSTM-based temporal memory improves out-of-sample trading robustness over a flat (memoryless) PPO baseline by capturing partial observability (POMDP). * **H2:** Replacing raw-return rewards with a **Differential Sharpe Ratio (DSR)** reward improves the agent's **risk-adjusted** performance **(higher Sharpe/Sortino, lower Max Drawdown)** compared to purely profit-driven rewards. * **H3:** Adding explicit turnover regularization reduces excessive trading (churn) and transaction costs while maintaining or improving the risk-adjusted performance of the DSR-guided agent.

## 4. Research Gap

> ***Zou et al. 2023, p. 9, Tables 2 and 3***

*A Novel Deep Reinforcement Learning Based Automated Stock Trading System Using Cascaded LSTM Networks*

Although deep reinforcement learning has shown promise for automated stock trading, existing approaches remain constrained by the noisy, unstable, and highly dynamic nature of financial market data. Prior machine learning models are also prone to overfitting, which reduces their generalization ability in real trading environments. In addition, reinforcement learning methods originally developed for gaming are not directly adaptable to financial data with low signal-to-noise ratios and uneven market behavior, which leads to performance limitations. Even in the proposed cascaded LSTM-PPO framework, further improvement still depends on unresolved issues such as the need for larger training datasets and more effective reward functions to improve stability and control pullback risk. Therefore, a clear research gap remains in developing more robust deep reinforcement learning trading systems that can better handle noisy financial data, generalize reliably, and achieve improved risk-adjusted performance

---

## 1. Introduction

In recent years, more and more institutional and individual investors are using machine learning and deep learning methods for stock trading and asset management, such as stock price prediction using Random Forests, Long Short-Term Memory(LSTM) Neural Networks or Support Vector Machines[1], which help traders to get well-performing online strategies and obtain higher returns than strategies using only traditional factors[2][3][4] .

However, there are three main limitations of machine learning methods for stock market prediction: (i) financial market data are filled with noise and are unstable, and also contain the interaction of many unmeasurable factors. Therefore, it is very difficult to take into account all relevant factors in complex and dynamic stock markets[5][6][7]. (ii) Stock prices can be influenced by many other factors, such as political events, the behavior of other stock markets or even the psychology of investors[8]. (iii) Most of the methods are performed based on supervised learning and require training sets that are labeled with the state of the market, but such machine learning classifiers are prone to suffer from overfitting, which reduces the generalization ability of the model[9].

Fundamental data from financial statements and other data from business news, etc. are combined with machine learning algorithms that can obtain investment signals or make predictions about the prospects of a company[10][11][12][13] to screen for good investment targets. Such an algorithm solves the problem of stock screening, but it cannot solve how to allocate positions among the investment targets. In other words, it is still up to the trader to judge the timing of entry and exit.

---

## 5. Conclusion

In this paper, we propose a PPO model using cascaded LSTM networks and compare the ensemble strategy in Yang[14] as a baseline model in the US, Chinese, Indian and UK market, respectively. The results show that our model has a stronger profit-taking ability, and this feature is more prominent in the Chinese market. However, according to the risk-return criterion, our model is exposed to higher pullback risk while obtaining high returns. Finally, we believe that the strengths of our model can be more fully exploited in markets where the overall trend is smoother and less volatile, as returns can be more consistently accumulated in such an environment (such as the A-share market in China in recent years). This suggests that there is indeed a potential return pattern in the stock market, and the LSTM as a time-series feature extractor plays an active role. Also, it shows that the Chinese market is a suitable environment for developing quantitative trading.

In the subsequent experiments, improvements can be made in the following aspects: (i) The amount of training data. The training of PPO requires a large amount of historical data to achieve good learning results, so expanding the amount of training data may help to improve the results. (ii) Reward function. Some improved reward functions for stock trading have emerged, which can enhance the stability of the algorithm. For example, we can refer to the Sharpe ratio approach to weigh risk and return to control pullback. The numerator is the mean of the return series and the denominator is the standard deviation of the return series. At the same time, if the reward signal has a very large or very small range, it can cause numerical instabilities during training. In this case, you can normalize the reward function to a smaller range (e.g., between -1 and 1) to make it more stable.

---

> ***Huang et al. 2024, p. 1 and p. 8***

*A Self-Rewarding Mechanism in Deep Reinforcement Learning for Trading Strategy Optimization*

**Huang et al. (2024)** investigate adaptive/self-rewarding mechanisms, showing that reward design can substantially affect trading performance and mitigate risk.

- Existing DRL trading systems mostly use static, manually designed reward functions, which do not adapt well to unstable and changing financial markets

- The unresolved problem is how to build a **dynamic reward mechanism** that can adjust during learning and remain responsive to market changes
- This paper addresses that gap by proposing a self-rewarding RL framework that combines expert labels with learned reward prediction

### 3.2. Self-Rewarding Mechanism

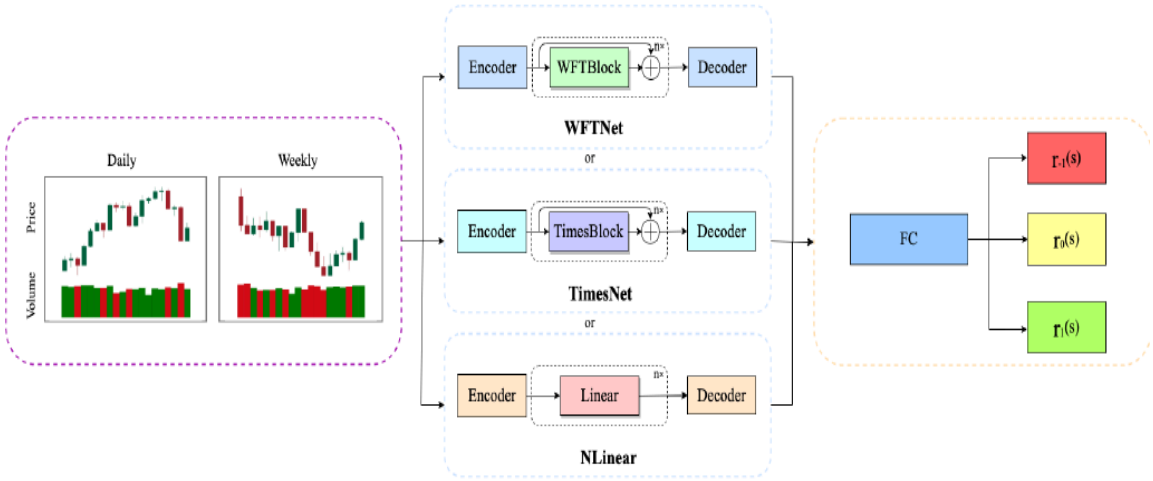
In traditional RL, the reward function is often manually designed and fixed, which can be limiting in dynamic environments such as financial markets. This subsection introduces a novel self-rewarding mechanism tailored for deep reinforcement learning, aimed to enhance the performance of optimized single-asset trading strategies. A self-rewarding mechanism allows the agent to adapt its reward function based on the environment and learning process, making it more flexible and responsive to changes. The RL agent can dynamically generate its own reward signals, instead of relying solely on predefined, static reward functions provided by human experts. As shown in Figure 2, the mechanism employs a training method similar to regression analysis. It assigns reward values to various states based on potential actions, identifies the highest rewards from these actions, and updates the learning based on the mean-square error with respect to the expert-defined rewards. The proposed method uses supervised learning to combine the insights of experienced market practitioners with advanced time series feature extraction networks, such as TimesNet [54], NLinear [55], and WFTNet [56]. These networks approximate a self-rewarding function, predict reward values, and then use expert-defined reward values as labels. Training through a backpropagation process guided by mean square error computation helps improve the accuracy of reward prediction.

However, a central challenge in reinforcement learning is designing reward functions that are both dynamic and effective. Reward functions, which are a core component of reinforcement learning, have a decisive impact on the learning process and the development of the final strategy. Traditional reinforcement learning methods typically rely on predefined static reward functions [1–10]. Although these approaches can achieve the desired goal to some extent, their limitations become apparent when faced with unstable and complex environments such as financial markets. Predefined reward functions often lack sufficient

/doi.org/10.3390/math12244020 https://www.mdpi.com/journal/mathematics



flexibility, making it difficult to adapt to dynamic changes in the environment, thus limiting the performance and adaptability of the algorithm.




> ***Millea 2021, p. 21, Section 12.1***

*Deep Reinforcement Learning for Trading—A Critical Survey*

Despite increasing interest in deep reinforcement learning for financial trading, the literature remains methodologically inconsistent and difficult to compare across studies. Existing works frequently use different datasets, different time periods, and different preprocessing techniques, which makes it difficult to determine whether reported performance differences are caused by the learning algorithm itself or by variation in the input data. In addition, only a limited number of studies examine model performance across multiple market types, leaving the generalizability of DRL-based trading systems insufficiently understood. Reproducibility is further weakened by the limited availability of source code and the incomplete reporting of implementation details such as hyperparameters and network architecture. Therefore, an important research gap remains in developing more standardized, comparable, and reproducible evaluation practices for DRL-based trading research

### 12.1. Common Ground

- Many papers use convolutional neural networks to (at least) preprocess the data.
- A significant number of papers concatenate the last few prices as a state for the DRL agent (this can vary from ten steps to hundreds).
- Many papers use the Sharpe ratio as a performance measure.
- Most of the papers deal with the portfolio management problem.
- Very few papers consider all three factors: transaction cost, slippage (when the market is moving so fast that the price is different at the time of the transaction than at the decision time), and spread (the difference between the buy price and sell price). The average slippage on Binance is around 0.1% https://blog.bybit.com/en-us/insights/a-comparison-of-slippage-in-different-exchanges/, accessed on 5 Octobere 2021, while the spread is around 1%, with the maximum even being 8% (see Figure 9).

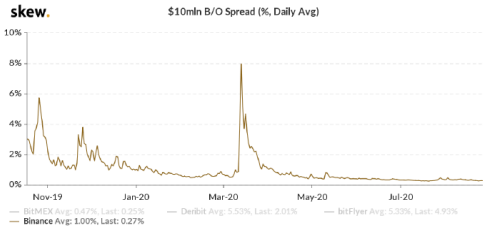


Figure 9. The Bitcoin spread on Binance.

*12.2. Methodological Issues*

- Inconsistency of data used: Almost all papers use different datasets, so it is hard to compare between them. Even if using the same assets (such as Bitcoin), the periods used are different, but in general, the assets are significantly different.
- Different preprocessing techniques: Many papers preprocess the data differently and thus feed different inputs to the algorithms. Even if algorithms are the same, the results might be different due to the nature of the data used. One could argue that this is part of research, and we agree, but the question remains: how can we know if the increased performance of one algorithm is due to the nature of the algorithm or the data fed?
- Very few works use multiple markets (stock market, FX, cryptocurrency markets, gold, oil). It would be interesting to see how some algorithms perform on different types of markets, if hyperparameters for optimal performance differ, and, if so, how they differ. Basically, nobody answers the question, how does this algorithm perceive and perform on different types of markets?
- Very few works have code available online. In general, the code is proprietary and hard to reproduce due to the missing details of the implementation, such as hyperparameters or the deep neural network architecture used.

---

**[Liu et al. 2024, p. 3, 11, 27]**

*Dynamic datasets and market environments for financial reinforcement learning*

Despite progress in financial reinforcement learning, existing studies still rely heavily on historical backtesting environments that may not represent real market conditions adequately. This creates a **simulation-to-reality** gap, where strong backtest results do not necessarily translate into robust live trading performance. The literature identifies several unresolved data-centric challenges, including **survivorship bias, low signal-to-noise ratio**, and **model overfitting**, which continue to limit the reliability and real-world deployment of FinRL agents

### 3.2 Major challenges

Training and testing environments based on historical data may not simulate real markets accurately due to the *simulation-to-reality gap* (Dulac-Arnold et al., 2021, 2019), and thus a trained agent cannot be directly deployed in real-world markets. We summarize the main FinRL challenges as follows:

- **Low signal-to-noise ratio (SNR)**: Data from different sources may contain large noises (Wilkman, 2020) such as random noise, outliers, etc. These cause the collected dataset with a low signal-to-noise ratio. It is challenging to identify alpha signals or build smart beta indices using such noisy datasets. FinRL-Meta's data curation pipeline holds data cleaning and feature engineering, which reduce noise and extract useful signals from the raw data.
- **Survivorship bias of historical market data**: Survivorship bias is caused by a tendency to focus on existing stocks and funds without consideration of those that are delisted (Brown et al., 1992). It could lead to an overestimation of stocks and funds, which will mislead the agent.
- **Model overfitting**: Existing research papers mainly report backtesting results. It is highly possible that authors are tempted to tune hyper-parameters of the chosen algorithm and retrain the agent multiple times[5] to obtain better backtesting results, resulting in model overfitting (Gort et al., 2023; De Prado, 2018). This might lead to big trouble during real-time trading. In FinRL-Meta, we use dynamic datasets to control the training–testing-trading window, which discards the outdated data and fetchs latest data. That prevents models from overfitting to the old data that are less influential.
- **Delay**: Financial markets have delays due to data transmission, reward feedback, and actuation. The true reward may be based on the users' interaction with the markets, which may take several days. As the delay increases, the performance of deep reinforcement learning decreases.
- **Partial observation**: The full observability assumption of financial markets can be extended to partial observation (the underlying states cannot be directly observed),

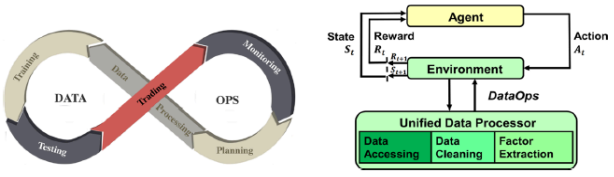



Fig. 1 DataOps paradigm (left) and FinRL-Meta (right)

However, building near-real market environments for financial reinforcement learning (FinRL) is difficult due to major factors such as low signal-to-noise ratio (SNR) of financial data, partial observation, reward delay, survivorship bias of historical data, and model overfitting. Such a *simulation-to-reality gap* (Dulac-Arnold et al., 2021, 2019) degrades the performance of DRL strategies in real markets. A good backtest performance does not necessarily reflect the actual trading performance since the financial dataset could be susceptible to flaws like missing data, noise, and anomalies.

Data-centric AI (Whang et al., 2023; Zha et al., 2023, 2023) is a new trend that appeals to shift the focus from model to data. It focuses on the systematic engineering of data in building AI systems, which can lead to better model behaviors in the real world (Zha et al., 2023). When training DRL agents, if the data is susceptible to flaws, even if our model/strategy can have a good backtest performance, it will be hard to tell whether the model can actually perform well in the real deployment or is just a result of overfitting the training dataset. Therefore, building a standard workflow to ensure data quality is imperative to foster reliable market environments and benchmarks and motivate the research and indus-

### 5.2.2 Trading in real time

Practical tasks like stock trading and cryptocurrency trading suffer from the false positive issue due to overfitting, where an agent might perform well on testing data but not on real-world markets. Since backtesting has problems of information leakage and overfitting, its results is not persuasive to show the quality of trained models. Thus, we propose to deploy DRL agent on paper trading.

Figure 12 shows our "training–testing-trading" pipeline. In a window, there are $N$ days' data for training and $S$ days' data for testing. At the end of a window, we perform paper trading for 1 day. Note that we always retrain the agent using $N + S$ days of training and testing data together. Then, we roll the window forward by 1 day ahead and perform the above steps for a new window. Paper trading is always carried out for 1 day. Therefore, $D$ windows correspond to $D$ trading days.

Alg. 1 summarizes the pipeline of paper trading. For $D$ trading days from 0 to $D - 1$, we keep doing the following three steps:

---

**[Wang & Liu (2025), Page 3 and Page 4, section 2.3]**

*Risk-Sensitive Deep Reinforcement Learning for Portfolio Optimization*

- Finally, **Wang & Liu (2025)** demonstrate the benefits of adaptive risk-sensitive policies under changing market conditions. They highlight that prior deep reinforcement learning research has focused predominantly on **equities and forex**, leaving commodity futures underexplored. Specifically, previous commodity-based DRL studies were limited in **adaptive agent-switching mechanisms or lacked portfolio-level optimization**, failing to fully address the distinctive structural volatility and supply-shock complexity of petroleum futures.
- The paper also identifies a second, more practical gap: even its own proposed framework does not yet explicitly model roll mechanics, expiration effects, or liquidity constraints, which limits real-world deployment.

*2.3. Commodity Futures Trading*

Recent studies have increasingly demonstrated the potential of deep reinforcement learning (DRL) in commodity futures markets:

- (Du et al., 2023) applied variational mode decomposition to Brent crude oil futures, integrating the decomposed features into an ensemble DRL framework. While effective in capturing non-stationary components, their model lacked adaptive agent switching mechanisms to dynamically adjust to changing market conditions.
- (Massahi & Mahootchi, 2024) developed a GRU-enhanced Deep Q-Network (DQN) model to address futures contract execution challenges. Although their approach improved intra-contract trading, it was primarily limited to managing single-contract positions and did not extend to portfolio-level optimization across multiple futures instruments.

In contrast, much of the earlier literature has focused on the application of DRL to equity and foreign exchange (FX) markets. For instance, (Moody & Saffell, 2001) introduced a recurrent reinforcement learning approach for stock trading, demonstrating its potential to model temporal dependencies in equity markets. Similarly, (Deng et al., 2016) explored deep learning architectures for financial signal representation and trading, providing a foundation for applying neural networks to complex financial time series forecasting problems.

While these prior works have advanced DRL applications across different financial instruments, the unique characteristics of commodity futures—including non-stationary price behavior, seasonality, and abrupt regime shifts—pose additional challenges that require models capable of learning long-range temporal dependencies. This motivates the incorporation of Transformer-based architectures, which have demonstrated superior performance in sequence modeling tasks across various domains.

---

### Combined Research Gap

Taken together, the reviewed studies address important but largely **separate aspects** of DRL-based financial trading:

- **Zou et al. (2023)** demonstrate the benefit of **LSTM-based temporal memory** for PPO trading by using historical sequences to capture temporal information. However, their framework mainly relies on **return-based rewards**, which do not directly account for risk measures such as variance or drawdown.
- **Huang et al. (2024)** address the limitation of **static reward functions** by introducing a self-rewarding mechanism that combines expert labels with learned reward prediction. This shows the importance of reward design, but the approach **replaces the reward mechanism** rather than isolating the effect of a mathematically defined risk-adjusted reward such as the Differential Sharpe Ratio (DSR).
- **Millea (2021)** highlights the problem of **inconsistent experimental settings** in DRL trading research, where studies often use different datasets, time periods, preprocessing methods, friction assumptions, and evaluation procedures. This makes it difficult to determine whether performance improvements come from the algorithm or from differences in the experimental setup.
- **Liu et al. (2024)** address the need for more realistic and reproducible financial RL experiments through **dynamic datasets, realistic market environments, and rolling training-testing-trading evaluation**. However, their main contribution is focused on the **environment and evaluation pipeline**, rather than isolating the contribution of advanced risk-aware reward mechanisms.
- **Wang & Liu (2025)** demonstrate **adaptive risk-sensitive decision-making** under changing market conditions. However, their framework does not separately isolate the contributions of **temporal memory, reward shaping, and turnover control** within a single controlled ablation.

### Overall Gap

These studies therefore motivate three important directions:

**(1) temporal memory, (2) risk-aware reward design, and (3) turnover regularization.**

However, **among the studies reviewed for this project, we did not find a controlled experimental framework that isolates these mechanisms incrementally within the same PPO trading environment**. In particular, their separate contributions are not systematically decomposed under **identical data, cost assumptions, training conditions, and evaluation metrics**.

Therefore, this BTP-1 proposes a controlled four-stage ablation:

```
PPO → LSTM-PPO → LSTM-PPO + DSR → LSTM-PPO + DSR + Turnover
```

to empirically quantify the incremental effect of each mechanism on **return, risk-adjusted performance, drawdown, turnover, and cost-adjusted trading stability**.

## 5. Literature Review

| Paper | Main Idea | Dataset/Market | State | Action | Reward | Key Finding | Limitation Relevant to BTP-1 |
|---|---|---|---|---|---|---|---|
| **Millea (2021)** | Survey of DRL in trading & market mechanics. | Crypto / Stocks / FX | Various (OHLCV, LOB) | Discrete & Cont. | Sharpe, PnL, Sortino | Identifies major inconsistencies in data, friction, and evaluation. | Meta-analysis; highlights the need for strict ablation & friction-control. |
| **Zou et al. (2023)** | Cascaded LSTM-PPO for automated trading. | US, CN, IN, UK Stocks | 181-dim (Prices, MACD, RSI, etc.) | Discrete (Shares) | Net Return | LSTM feature extractor outperforms ensemble & flat methods (TW=30, HS=512). | Relies on absolute profit reward; blind to variance/drawdown risk. |
| **Liu et al. (2024)** | DataOps pipeline and dynamic walk-forward environments. | Dow 30, S&P 500, Crypto | Balances, Prices, Tech Indicators | Continuous (Weights) | Asset value change | RLOps standardizes environments, reducing simulation-to-reality gap. | Standard baselines lack advanced online risk-aware reward functions. |
| **Huang et al. (2024)** | Self-rewarding mechanism blending expert labels. | DJI, IXIC, SP500, HSI | OHLCV | Discrete | Sharpe, Min-Max | Self-rewarding significantly beats fixed formulaic rewards. | Replaces rather than regularizes reward; relies on predefined expert labels. |
| **Wang & Liu (2025)** | ART-DRL: Adaptive risk-sensitive DRL. | Equities | Market/Tech Features | Continuous | Adaptive Risk | Dynamically shifts risk sensitivity based on market regime. | Focuses on adaptive risk sensitivity but does not isolate the independent contribution of temporal memory versus reward shaping in a controlled ablation. |

## 6. Common Trading Environment

To ensure strict comparability, all four models will be trained and evaluated in the same simulated trading environment.

- **Data:** Daily OHLCV data for a fixed universe of (N) equities.
- **Decision Frequency:** One portfolio-rebalancing decision is made per trading day.
- **Transaction Costs:** A fixed pro
- **Data:** Daily OHLCV data for a fixed universe of $N$ equities.
- **Decision Frequency:** One portfolio-rebalancing decision is made per trading day.
- **Transaction Costs:** A fixed proportional transaction-cost rate $c_{\text{trans}}$ is applied to traded portfolio value.
- **Slippage:** A fixed proportional slippage assumption may be incorporated into the effective transaction-cost rate.
- **Portfolio Constraint:** Long-only allocation across $N$ stocks and an explicit cash component.
- **Evaluation:** Strict walk-forward training, validation (where required), and out-of-sample testing.

### POMDP Formulation

The daily trading problem is formulated as a Partially Observable Markov Decision Process (POMDP). The agent cannot observe all latent factors governing financial markets and therefore receives only a partial observation of the underlying environment.

We define the POMDP as:

$$\mathcal{P} = (\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \Omega, \gamma)$$

where:

- $S$ : is the underlying environment state space,
- $\mathcal{A}$ is the action space,
- $\mathcal{T}$ represents the environment transition dynamics,
- $\mathcal{R}$ is the reward function,
- $\Omega$ is the observation space,
- $\gamma \in (0, 1]$ is the PPO discount factor.

At time $t$ , the agent receives an observation $s_t \in \Omega$ containing only information available up to the current decision time. For M1, the policy operates directly on $s_t$ . For M2--M4, a sequence of observations is provided to an LSTM to construct a temporal representation $h_t$ .

**MDP / POMDP Formulation:** Because financial markets are heavily influenced by unobservable latent factors, a standard MDP $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$ is insufficient. The temporal information motivates a Partially Observable MDP (POMDP), formulated as $(\mathcal{O}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$ , where the agent receives observations $o_t \in \mathcal{O}$ and utilizes an RNN (LSTM) to maintain a hidden belief state $h_t$ approximating the true market state.

## 7. State / Observation Representation

The observation provided to the agent contains only information available at decision time $t$ , with all feature transformations computed causally to avoid look-ahead bias.

For each asset $i \in \{1, \ldots, N\}$ , define the per-asset feature vector:

$$f_{i,t} \in \mathbb{R}^F$$

where $F$ is the number of features per asset.

For each asset $(i \in \{1, \ldots, N\})$ , define the per-asset feature vector:

$$f_{i,t} = \left[ r_{i,t}, \frac{O_{i,t}}{C_{i,t}} - 1, \frac{H_{i,t}}{C_{i,t}} - 1, \frac{L_{i,t}}{C_{i,t}} - 1, \tilde{v}_{i,t}, \frac{RSI_{i,t}}{50} - 1 \quad MACD_{i,t}, \frac{EMA^{fast}_{i,t}}{C_{i,t}} - 1, \frac{EMA^{slow}_{i,t}}{C_{i,t}} - 1, \%B_{i,t}, BW_{i,t}, \frac{CCI_{i,t}}{100}, \frac{ADX_{i,t}}{100} \right]^\top \in \mathbb{R}^{13}$$

where $(r_{i,t})$ is the log return, $(\tilde{v}_{i,t})$ is the causally normalized volume, and the remaining terms represent price-relative and technical-indicator information.

The feature vector contains normalized return, price-relative, volume, and technical-indicator information, including:

- log return,
- OHLC price-relative features,
- normalized volume,
- RSI,
- MACD,
- EMA-relative features,
- Bollinger $\%B$ and bandwidth,
- CCI,
- ADX.

Raw price levels and raw portfolio/account dollar values are not directly provided to the agent.

The market-feature vector is obtained by concatenating the feature vectors of all (N) assets:

$$x_t = concat\left(f_{1,t}, f_{2,t}, \ldots, f_{N,t}\right) \in \mathbb{R}^{13N}$$

The complete observation additionally includes the current portfolio weights:

$$s_t = \left[x_t; w_t^{cur}\right] \in \mathbb{R}^{13N+N+1}.$$

The portfolio state is represented by the current portfolio-weight vector:

$$w_t^{cur} = \left[w_{1,t}^{cur}, \ldots, w_{N,t}^{cur}, w_{cash,t}^{cur}\right] \in \mathbb{R}^{N+1},$$

subject to:

$$w_{i,t}^{cur} \geq 0, \qquad w_{cash,t}^{cur} \geq 0,$$

$$\sum_{i=1}^{N} w_{i,t}^{cur} + w_{cash,t}^{cur} = 1.$$

The complete observation is therefore:

$$\boxed{s_t = \left[x_t; w_t^{cur}\right]}$$

with dimension:

$$s_t \in \mathbb{R}^{NF+N+1}.$$

Here:

- $N$ = number of stocks in the trading universe,
- $F$ = number of features per stock,
- $f_{i,t}$ = feature vector of asset $i$ at time $t$ ,
- $x_t$ = concatenated market-feature vector,
- $w_t^{cur}$ = current portfolio weights including cash,
- $s_t$ = complete observation available to the RL agent.

### Temporal Representation

For M1, only the current observation $s_t$ is provided to the policy.

For M2--M4, a historical observation window of length $W$ is constructed:

$$F_t = \left[s_{t-W+1}, \ldots, s_{t-1}, s_t\right]$$

where:

$$F_t \in \mathbb{R}^{W \times (NF+N+1)}.$$

The sequence $F_t$ is processed by an LSTM to produce the hidden representation:

$$(h_t, c_t) = \mathrm{LSTM}(F_t),$$

where $h_t \in \mathbb{R}^H$ is the hidden representation and $c_t \in \mathbb{R}^H$ is the LSTM cell state. $H$ denotes the LSTM hidden size.

For the initial BTP-1 configuration:

$$W = 30, \qquad H = 512.$$

The LSTM provides a learned temporal representation of the historical observations; it does not explicitly predict future prices.

## 8. Action Space

The action space $\mathcal{A}$ must ensure that portfolio weights are non-negative and sum to 1. Rather than predicting unbounded values and applying a post-hoc Softmax, we directly ground our continuous action space in the Dirichlet distribution formulation presented by **Yang et al. (2022)**.

- **Reference:** *Yang, H., Park, H., & Lee, K. (2022), "A Selective Portfolio Management Algorithm with Off-Policy Reinforcement Learning Using Dirichlet Distribution"*

The action $a_t$ is defined exactly as the target portfolio weights:

$$a_t \equiv w_t^{target}$$

where $w_{i,t}^{target} \geq 0$ and $\sum_{i=1}^{N} w_{i,t}^{target} + w_{cash,t}^{target} = 1$ .

**Dirichlet Parameterization:** Following Yang et al., the policy network outputs the concentration parameters $\alpha_t$ of a Dirichlet distribution. To ensure $\alpha_t > 0$ , we use an exponential mapping from the network's output logits $m_t$ :

$$m_t = W_a h_t + b_a$$

$$\alpha_{i,t} = \exp(m_{i,t})$$

The target weights are then sampled from the resulting Dirichlet distribution:

$$w_t^{target} \sim \text{Dirichlet}(\alpha_t)$$

### Addressing the Dirichlet Sparsity Limitation (Weight Thresholding)

A mathematical limitation of the standard Dirichlet distribution is that its support lies on the open simplex $(0, 1)$ . It mathematically cannot output an exact $0$ . In practical portfolio management, sparsity is critical: an agent must be able to hold $0\%$ of an underperforming stock to avoid continuous, microscopic rebalancing (e.g., adjusting a weight from $0.002$ to $0.001$ ) that bleeds capital through transaction fees.To enforce sparsity, prevent micro-churn, and make the continuous action space viable for real-world transaction costs, we apply a deterministic thresholding mask to the sampled weights before passing them to the portfolio environment.

1. Thresholding:Apply a minimum allocation boundary $\tau$ (e.g., $\tau = 0.01$ or $1\%$ ). If a generated weight falls below this threshold, it is forced to zero:

$$\tilde{w}_{i,t}^{target} = \begin{cases} 0, & \text{if } w_{i,t}^{target} < \tau \\ w_{i,t}^{target}, & \text{otherwise} \end{cases}$$

2. Renormalization:Re-normalize the remaining active weights to ensure the portfolio constraint $\sum w_i = 1$ is maintained:

$$w_{i,t}^{final} = \frac{\tilde{w}_{i,t}^{target}}{\sum_j \tilde{w}_{j,t}^{target}}$$

   The environment and transaction cost models will execute the portfolio rebalancing based exclusively on $w_t^{final}$ . This mathematically bridges the gap between the continuous, strictly positive probability density of the Dirichlet actor and the sparse allocation reality of financial markets.

The expected weight for each asset is naturally defined by the properties of the Dirichlet distribution:

$$\mathbb{E}[w_{i,t}^{target}] = \frac{\alpha_{i,t}}{\sum_j \alpha_{j,t}}$$

Here:

- $a_t$ = RL action at time $t$ ,
- $w_t^{target}$ = target portfolio weights selected by the agent,
- $w_{i,t}^{target}$ = target weight of stock $i$ ,
- $w_{cash,t}^{target}$ = target cash weight,
- $\alpha_t$ = Dirichlet concentration-parameter vector,
- $m_t$ = unconstrained actor output,
- $W_a, b_a$ = parameters of the actor's final layer,
- $h_t$ = LSTM hidden representation for M2--M4.

The Dirichlet formulation is motivated by Yang et al. (2022), who use the Dirichlet distribution to model portfolio allocations on the simplex. Their formulation is adapted here to the on-policy PPO setting used in BTP-1.

> ***Yang et al. 2022*, Section 3.3, Equations 13-16**
>
> *A Selective Portfolio Management Algorithm with Off-Policy Reinforcement Learning Using Dirichlet Distribution*

*3.3. Distribution of a Portfolio*

In this section, we define a distribution of a portfolio. Portfolio vector $\mathbf{w}_t$ has innate sum-to-one constraints. In other words, a set of valid portfolio vectors is equivalent to an $N-1$-dimensional simplex. Hence, to assign a distribution of a portfolio, we employ the Dirichlet distribution, which is a well-known parametric distribution over a simplex. Dirichlet distributions with concentration parameters $\alpha_1, \alpha_2, \ldots \alpha_K > 0$ have a multivariate probability density function as follows,

$$f(x_1, \ldots, x_K; \alpha_1, \ldots, \alpha_K) = \frac{1}{B(\alpha)} \prod_{i=1}^{K} x_i^{\alpha_i - 1}. \tag{13}$$

where $B(\alpha)$ is the beta function defined as,

$$B(\alpha) := \frac{\prod_{i=1}^{K} \Gamma(\alpha_i)}{\Gamma(\sum_{i=1}^{K} \alpha_i)}, \quad \alpha = (\alpha_i, \ldots, \alpha_K) \tag{14}$$

$$\Gamma(x) := \int_0^\infty t^{x-1} e^{-t} dt. \tag{15}$$

Note that random vector $X$ sampled from Dirichlet distribution is an element in the $K-1$ simplex. Hence, $\sum_{i=1}^{K} X_i = 1$ and $X_i \in [0,1]$ hold. This property is suitable for representing a portfolio vector. By using a Dirichlet distribution, we can define a stochastic policy over action space. We would like to emphasize that this paper is the first to employ the Dirichlet distribution to represent a stochastic policy over a portfolio vector.

predicted by the score network.

Based on $\{m_i\}_{i=1}^{K}$, the Dirichlet policy can be defined. Since the characteristics of the Dirichlet distribution are fully determined by the concentration parameters; we define the concentration parameters as a function of scores. Specifically, since the concentration parameters must be positive, all parameters are computed as the exponential of the scores. Finally, to consider a cash asset, we add a cash bias. Hence, the distribution policy is defined as

$$a = \mathbf{w}_t - D, \quad D \sim \mathrm{Dir}([1, e^{m_2}, e^{m_3}, \ldots, e^{m_K}]), \tag{16}$$

where 1 indicates a cash bias and $D$ indicates the desired portfolio. By using this policy, the

- ***Purpose:*** *Establishes the literature-grounded mathematical mechanism for learning valid, continuous portfolio weights via Dirichlet concentration parameters. (Note: While Yang et al. apply this in an off-policy framework, we adopt the continuous action-space parameterization for our on-policy PPO).*

## 9. Portfolio Dynamics and Cost Model

The portfolio environment converts the target allocation generated by the agent into actual portfolio rebalancing, applies transaction costs, and updates portfolio wealth.

### 9.1 Current and Target Portfolio Weights

The current portfolio before rebalancing is:

$$w_t^{cur} = [w_{1,t}^{cur}, \ldots, w_{N,t}^{cur}, w_{cash,t}^{cur}]$$

The agent selects:

$$w_t^{target} = [w_{1,t}^{target}, \ldots, w_{N,t}^{target}, w_{cash,t}^{target}]$$

For each traded stock, the change in allocation is:

$$\Delta w_{i,t} = w_{i,t}^{target} - w_{i,t}^{cur}$$

Here:

- $\Delta w_{i,t} > 0$ indicates an increase in allocation,
- $\Delta w_{i,t} < 0$ indicates a decrease in allocation,
- $\Delta w_{i,t} = 0$ indicates no change.

### 9.2 Transaction Cost

Transaction cost is modeled as proportional to the absolute amount reallocated:

$C_t = c_{trans} V_{t-1} \sum_{i=1}^{N} |\Delta w_{i,t}|$

where:

- $C_t$ = monetary transaction cost at time t,
- $c_{trans}$ = proportional transaction-cost rate,
- $V_{t-1}$ = portfolio value immediately before rebalancing,
- $\Delta w_{i,t}$ = change in stock i's portfolio weight.

The cash component is not separately charged; cash is the residual portfolio allocation after stock rebalancing.

### 9.3 Portfolio Wealth Evolution

After paying transaction costs, the portfolio evolves according to the target stock allocation and realized stock returns:

$$V_t = (V_{t-1} - C_t) \left[ \sum_{i=1}^{N} w_{i,t}^{target} (1 + R_{i,t}) + w_{cash,t}^{target} \right]$$

Here:

- $V_t$ = portfolio value at the end of period $t$ ,
- $R_{i,t}$ = realized return of stock $i$ over period $t$ ,
- $w_{i,t}^{target}$ = target weight allocated to stock $i$ ,
- $w_{cash,t}^{target}$ = target cash allocation.

The cash component is assumed to have zero return over the daily holding period for BTP-1; therefore, no risk-free-rate term is included in the portfolio wealth equation.

### 9.4 Net Portfolio Return

The realized net portfolio return is:

$$R_{net,t} = \frac{V_t - V_{t-1}}{V_{t-1}}$$

Because transaction costs are already deducted when computing $V_t$ , they are not subtracted again from $R_{net,t}$ .

Thus, $R_{net,t}$ is the single accounting measure of realized portfolio growth after modeled trading frictions and is used as the direct reward for M1 and M2 and as the return input to the DSR calculation for M3 and M4.

## 10. Four-Model Ablation: Detailed Mathematical Modelling

### M1 — PPO Baseline

- **Architecture:** Memoryless feed-forward Multi-Layer Perceptron (MLP) for both the actor $\pi_\theta(a_t|s_t)$ and critic $V_\phi(s_t)$ networks.
- **State Input:** Only the instantaneous current step observation $s_t$ .
- **Reward:** Direct net return, $r_t = R_{net,t}$ **(Liu et al. 2024, §3.1, p. 10)**.
- **Objective:** Standard Generalized Advantage Estimation (GAE) where $\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + \dots$ and $\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$ . The actor is updated using the clipped surrogate objective:

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[ \min \left( \rho_t(\theta)\hat{A}_t, \text{clip}\left(\rho_t(\theta), 1-\epsilon, 1+\epsilon\right)\hat{A}_t \right) \right]$$

### M2 — LSTM-PPO (+ Temporal Memory)

- **Mechanism Added:** Temporal memory (LSTM) to handle POMDP nature of financial data.
- **Architecture:** Observation window $F_t = [s_{t-W+1}, \dots, s_t]$ is passed through an LSTM. The hidden state $h_t$ and cell state $c_t$ update recursively:

$$h_t, c_t = LSTM_{cell}(s_t, h_{t-1}, c_{t-1})$$

- **Conditioning:** The policy and value functions are now conditioned on the hidden representation: $\pi_\theta(a_t|h_t)$ and $V_\phi(h_t)$ . Note that the LSTM does not explicitly predict future prices; it forms a learned temporal representation $h_t$ .

- **Parameters:** Rather than arbitrary tuning, we strictly adopt the architecture validated by **Zou et al. (2023, §4.5.1 & §4.5.2, p. 9)**: Time Window ( $W$ ) = 30, Hidden Size (HS) = 512.
- **Reward:** Direct net return, $r_t = R_{net,t}$ .

> *Zou et al. 2023, "A Novel DRL Based Automated Stock Trading System...", p. 9*

Table 2: Comparison of different time windows in LSTM

| | TW=5 | TW=15 | TW=30 | TW=50 |
|---|---|---|---|---|
| CR | 32.69% | 51.53% | **90.81%** | 68.74% |
| MER | 68.27% | 63.46% | **113.50%** | 79.32% |
| MPB | 58.93% | **24.75%** | 46.51% | 37.01% |
| APPT | 18.29 | 21.77 | **35.27** | 23.31 |
| SR | 0.2219 | 0.7136 | **1.1540** | 0.9123 |

Table 3: Comparison of different hidden sizes of LSTM in PPO

| | HS=128 | HS=256 | HS=512 | HS=1024 | HS=512*2 |
|---|---|---|---|---|---|
| CR | 69.94% | 72.58% | **90.81%** | 56.27% | 89.13% |
| MER | 84.29% | 82.32% | **113.50%** | 60.64% | 92.34% |
| MPB | 38.57% | **29.92%** | 46.51% | 30.39% | 58.31% |
| APPT | 28.07 | 30.79 | **35.27** | 23.96 | 33.26 |
| SR | 0.9255 | 1.0335 | **1.1540** | 0.8528 | 0.8447 |

- ***Purpose:*** *Justifies the direct adoption of the LSTM baseline variables without needing to re-tune from scratch.*

### M3 — LSTM-PPO + DSR

- **Mechanism Added:** Dense, risk-aware online reward. Standard profit rewards (M1 & M2) are blind to variance and drawdown risk.
- **Formulation:** Standard Sharpe requires a full episode to compute, causing sparse delayed rewards. Following **Millea (2021, §5.1.2, p. 8, Eq. 6 & 7)**, we use exponential moving estimates for the first moment ( $A_t$ ) and second moment ( $B_t$ ) of the net returns $R_{net,t}$ :

$$A_t = A_{t-1} + \eta(R_{net,t} - A_{t-1})$$

$$B_t = B_{t-1} + \eta(R_{net,t}^2 - B_{t-1})$$

Expanding the Sharpe ratio via a Taylor series yields the online DSR step-reward:

$$D_t = \frac{B_{t-1}\Delta A_t - \frac{1}{2}A_{t-1}\Delta B_t}{(B_{t-1} - A_{t-1}^2 + \varepsilon)^{3/2}}$$

* **Parameters:** $\eta \in (0, 1]$ is the DSR moving-average adaptation rate, strictly distinct from the PPO discount factor $\gamma$ . ** $\varepsilon$ : **A small numerical-stability constant (e.g., $10^{-8}$ ) added to the DSR denominator to prevent instability when the estimated variance approaches zero.** * Reward:** $r_t = D_t$ .

> *Millea 2021, "Deep Reinforcement Learning for Trading—A Critical Survey", p. 8, Section 5.1.2*

5.1.2. Differential Sharpe Ratio

In an online learning setting, as is the case for RL (if using this measure as a reward shaping mechanism), a different measure is needed, which does not require all the returns to obtain the expectation over the returns, which are obviously not available in an online setting. To overcome this limitation, [45] has derived a new measure given by

$$D_t = \frac{B_{t-1}\Delta A_t - \frac{1}{2}A_{t-1}\Delta B_t}{(B_{t-1} - A_{t-1}^2)^{3/2}} \quad (6)$$

where $A_t$ and $B_t$ are exponential moving estimates of the first and second moment of $R_t$, where $R_t$ is the estimate for the return at time $t$ ($R_T$ is the usual return used in the standard Sharpe ratio, with $T$ being the final step of the evaluated period), so a small $t$ usually refers to the current time-step. $A_t$ and $B_t$ are defined as

$$\begin{aligned} A_t &= A_{t-1} + \eta\Delta A_t = A_{t-1} + \eta(R_t - A_{t-1}) \\ B_t &= B_{t-1} + \eta\Delta B_t = B_{t-1} + \eta(R_t^2 - B_{t-1} \end{aligned} \quad (7)$$

- ***Purpose:*** *Establishes the exact mathematical foundation for the M3 risk-aware reward mechanism.*

### M4 — LSTM-PPO + DSR + Turnover Regularization

- **Mechanism Added:** Action-friction control to regularize churn.
- **Formulation:** DSR mathematically incentivizes the agent to capture tiny, high-Sharpe anomalies, leading to high-frequency action oscillation ("churn"). In live markets, slippage destroys these theoretical returns. To strictly isolate friction-control from risk-sensitivity (M3 → M4

comparison), the turnover penalty must be additive.

$$r_t = D_t - \lambda_{turnover} \sum_{i=1}^{N} (w_{i,t}^{target} - w_{i,t}^{cur})^2$$

- **Note:** This penalty $\lambda_{turnover}$ only punishes the RL *reward signal* to discourage churning. The actual portfolio simulation already accounts for true transaction costs in $R_{net,t}$ . Comparing M3 to M4 will explicitly test the hypothesis that regularizing action outputs stabilizes the LSTM memory mechanism.

## 11. Controlled Experimental Design

To isolate the incremental contribution of each mechanism, all models will be evaluated under the same experimental conditions:

- Identical daily datasets and feature sets.
- Identical walk-forward folds, so every model is exposed to the same market periods and regimes.
- Identical transaction-cost and slippage assumptions.
- Identical evaluation metrics and evaluation protocol.
- Comparable training budgets and multiple fixed random seeds across Python, NumPy, PyTorch, and the RL environment.

**Ablation Logic:**

- `M1 → M2` is intended to isolate the contribution of **temporal memory**.
- `M2 → M3` is intended to isolate the contribution of **DSR-based risk-aware reward shaping**.
- `M3 → M4` is intended to isolate the contribution of **turnover regularization**.

## 12. Walk-Forward Backtesting

Financial time series are non-stationary, and model performance can depend strongly on the market period used for training and testing. A single static train/test split provides only one out-of-sample period and may not adequately evaluate robustness across changing market conditions. Therefore, we will use a strict walk-forward evaluation methodology:

1. Train on window $T_0 \rightarrow T_k$ .
2. Validate, if required for hyperparameter selection, on $T_k \rightarrow T_{k+m}$ .
3. Test out-of-sample on $T_{k+m} \rightarrow T_{k+m+n}$ .
4. Roll the entire window forward by $n$ days and repeat.

> ***Liu et al. 2024* "Dynamic datasets and market environments...", p. 13, Figure 5**

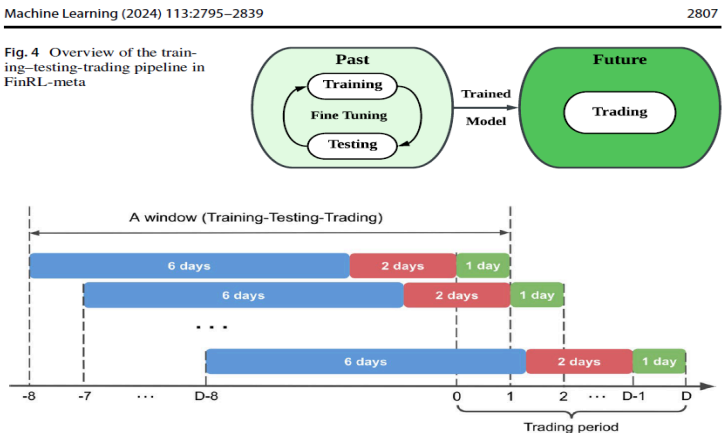



Fig. 5 A rolling window of training–testing-trading pipeline with dynamic dataset

- ***Purpose:*** *Provides literature grounding for the strict walk-forward methodology preventing look-ahead bias.*

## 13. Evaluation Metrics

The final out-of-sample arrays will be concatenated and evaluated using standard quantitative finance metrics to properly assess H2 and H3: * **Cumulative Return (CR):** $CR = \frac{P_{end} - P_0}{P_0}$ * **Annualized Return (AR)** * **Sharpe Ratio (SR):** $SR = \frac{\mathbb{E}[R_P] - R_f}{\sigma_P}$ * **Sortino Ratio** * **Maximum Drawdown (MDD):** $MDD = \max_t \left( \frac{Peak_t - V_t}{Peak_t} \right)$

- **Annualized Volatility**
- **Cumulative Transaction Cost**

Turnover and transaction cost are particularly important diagnostics for evaluating the effect of M4.

> ***Huang et al. 2024* "A Self-Rewarding Mechanism...", p. 12, Section 4.2**

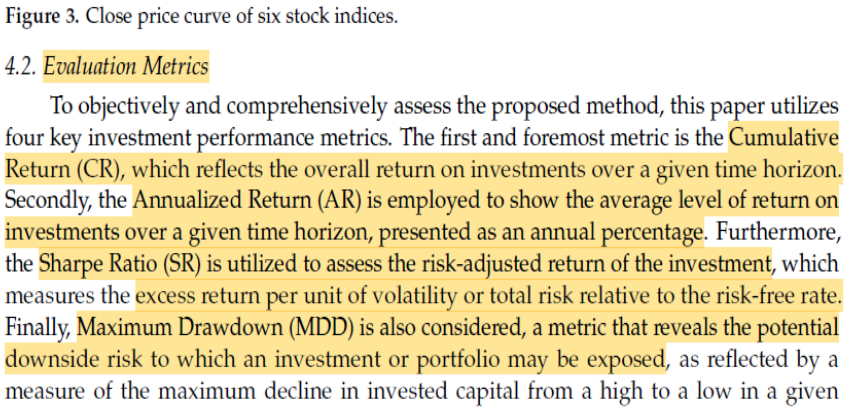


Figure 3. Close price curve of six stock indices.

4.2. *Evaluation Metrics*

To objectively and comprehensively assess the proposed method, this paper utilizes four key investment performance metrics. The first and foremost metric is the Cumulative Return (CR), which reflects the overall return on investments over a given time horizon. Secondly, the Annualized Return (AR) is employed to show the average level of return on investments over a given time horizon, presented as an annual percentage. Furthermore, the Sharpe Ratio (SR) is utilized to assess the risk-adjusted return of the investment, which measures the excess return per unit of volatility or total risk relative to the risk-free rate. Finally, Maximum Drawdown (MDD) is also considered, a metric that reveals the potential downside risk to which an investment or portfolio may be exposed, as reflected by a measure of the maximum decline in invested capital from a high to a low in a given

- ***Purpose:*** *Academic justification of the standard financial evaluation metrics.*

## 14. Experimental Matrix

### Controlled Ablation Summary

| Model | Architecture | Reward | Memory | Turnover Regularization | Main Question |
|---|---|---|---|---|---|
| **M1** | PPO (MLP) | Net Return | None | None | Baseline performance without memory, risk shaping, or action regularization |
| **M2** | LSTM-PPO | Net Return | LSTM with window $W$ selected during validation | None | Does temporal memory improve robustness and adaptability? |
| **M3** | LSTM-PPO | DSR | LSTM | None | Does risk-aware reward shaping improve risk-adjusted performance? |
| **M4** | LSTM-PPO | DSR | LSTM | $\lambda_{\text{turn}} \sum_i (w_{i,t}^{target} - w_{i,t}^{cur})^2$ | Does turnover regularization reduce excessive allocation changes and trading costs while preserving risk-adjusted performance? |

**Execution Table:**

| Dataset | Train/Test Windows | Roll Window | Transaction Cost | Seeds | Metrics |
|---|---|---|---|---|---|
| [TODO] | [TODO] | [TODO] | [TODO / literature-supported] | Multiple fixed seeds (e.g. 42, 100, 999 ~ *tentative*) | CR, AR, SR, Sortino, MDD, Volatility, Turnover, Cost |

---

Hyperparameter Tuning Protocol (Isolating $\eta$ and $\lambda_{\text{turn}}$ )To prevent data leakage, the DSR adaptation rate ( $\eta$ ) and turnover regularization coefficient ( $\lambda_{\text{turn}}$ ) will not be arbitrarily set. They will be selected via grid search exclusively on the validation folds ( $T_k \rightarrow T_{k+m}$ ) prior to out-of-sample testing. $\eta$ (Memory Decay): Will be searched over $[0.01, 0.05, 0.1]$ , where $\eta = 0.05$ roughly corresponds to a 20-day half-life, aligning the risk calculation with a monthly trading horizon. $\lambda_{\text{turn}}$ (Friction Penalty): Will be tuned to ensure the penalty magnitude is proportional to the average step-level DSR gradient, preventing the regularization term from completely dominating the policy update or being ignored.

---

## 15. Expected Analysis

The goal of this analysis is not simply "highest return wins."

- **H1:** If supported, M2 should demonstrate improved robustness across out-of-sample folds compared with M1, potentially with changes in volatility and drawdown.

- **H2:** If supported, M3 should demonstrate improved risk-adjusted performance over M2, such as a higher Sharpe/Sortino Ratio and/or lower Maximum Drawdown, even if its Cumulative Return is not higher.
- **H3:** If supported, M4 should exhibit lower **Average Turnover** and **Cumulative Transaction Costs** than M3 while maintaining or improving risk-adjusted out-of-sample performance.
- Fold-by-fold performance will be analyzed to assess whether the models remain robust during high-volatility and adverse market periods.

## 16. Failure Analysis

We will proactively investigate and report failure modes. If a model performs poorly or becomes unstable, the analysis will examine:

- **High Turnover / Churn:** Does the agent make frequent or excessively large allocation changes?
- **Regime Sensitivity:** Does the model perform well in certain market regimes but deteriorate substantially in others?
- **Reward Instability:** Does the DSR calculation become numerically unstable, leading to NaNs or unstable training?
- **Seed Sensitivity:** Does performance vary substantially across different random seeds?

*Note: The observed failure modes will be used as evidence to motivate and refine the direction of BTP-2.*

## 17. BTP-2 / MTP Future Direction

**BTP-2 will be guided by the failure modes and empirical findings identified in BTP-1.** Rather than introducing advanced techniques arbitrarily, the next stage will address the specific limitations observed in the best-performing BTP-1 configuration.

A primary direction for BTP-2 is to move from portfolio-level allocation decisions toward **explicit asset-level trading decisions**. For each stock (i) at time (t), the agent may be designed to output:

$$a_{i,t} \in \{\mathrm{BUY}, \mathrm{HOLD}, \mathrm{SELL}\}.$$

The BTP-2 framework may also incorporate an auxiliary prediction task for future price movement or future return over a selected horizon:

$$y_{i,t}^{(H)} = \frac{P_{i,t+H} - P_{i,t}}{P_{i,t}}.$$

This would allow the model to study both **trading decisions** and **future market-movement prediction** within a unified framework.

Depending on the failure modes identified in BTP-1, further extensions may include regime-aware or adaptive risk-sensitive decision-making. More advanced approaches, such as multi-agent reinforcement learning or self-rewarding mechanisms, will be considered only if they provide a clear research justification based on the BTP-1 results.

## 18. Limitations

The proposed BTP-1 framework has several deliberate limitations.

First, the main ablation uses a **fixed trading universe and fixed number of assets (N)** across all four models. This is intentional because changing the asset universe between models would introduce an additional source of variation and weaken the controlled comparison. Consequently, the current flat concatenation architecture is not intended to support an arbitrary number of assets without modification.

Second, BTP-1 focuses on **daily-frequency equity trading** and therefore does not model intraday microstructure, order-book dynamics, latency, or high-frequency execution effects.

Third, the transaction-cost model captures proportional trading costs but does not fully reproduce all real-world market frictions, such as market impact, bid-ask spread dynamics, slippage variation, and liquidity constraints.

Fourth, the current state representation is based on engineered market and technical features. It does not explicitly incorporate richer information sources such as order-flow data, news, fundamentals, or alternative data.

Finally, the conclusions of BTP-1 will depend on the selected assets, market period, transaction-cost assumptions, and experimental design. Therefore, strong out-of-sample and cross-asset validation will be important before drawing broader conclusions about the generality of the learned trading policy.

## 19. Final Summary

This project proposes a controlled empirical ablation study progressing from PPO to LSTM-PPO, DSR-based reward shaping, and turnover regularization. By maintaining consistent datasets, trading universe, transaction-cost assumptions, training conditions, and walk-forward evaluation, the study aims to isolate the incremental contribution of temporal memory, risk-aware reward design, and turnover control.

The primary objective of BTP-1 is therefore not to develop a universally applicable trading agent, but to establish a **rigorous and reproducible understanding of how these components affect out-of-sample trading performance and failure modes**.

The resulting performance and failure analysis will provide an evidence-based basis for selecting the direction of BTP-2, including the transition toward asset-level BUY/HOLD/SELL decisions and future market-movement prediction where justified by the BTP-1 findings.

---

### Mathematical Notation and Parameters

| Symbol | Meaning |
|---|---|
| $t$ | Trading day / decision time |
| $i$ | Asset index |
| $N$ | Number of stocks in the trading universe |
| $F$ | Number of input features per asset |
| $f_{i,t}$ | Feature vector of asset $i$ at time $t$ , $f_{i,t} \in \mathbb{R}^F$ |
| $x_t$ | Concatenated market-feature vector, $x_t \in \mathbb{R}^{NF}$ |
| $w_t^{cur}$ | Current portfolio-weight vector including cash |
| $w_{i,t}^{cur}$ | Current portfolio weight of stock $i$ |
| $w_t^{target}$ | Target portfolio-weight vector selected by the agent |
| $w_{i,t}^{target}$ | Target portfolio weight of stock $i$ |
| $w_{cash,t}^{target}$ | Target cash allocation |
| $s_t$ | Complete agent observation at time $t$ |
| $W$ | Historical lookback-window length |
| $F_t$ | Sequence of the previous $W$ observations |
| $H$ | LSTM hidden-state dimension |
| $h_t$ | LSTM hidden representation |
| $c_t$ | LSTM cell state |
| $a_t$ | RL action, defined as $a_t \equiv w_t^{target}$ |
| $m_t$ | Unconstrained actor output before Dirichlet parameterization |
| $W_a, b_a$ | Parameters of the actor's final layer |
| $\alpha_t$ | Dirichlet concentration-parameter vector |
| $\alpha_{i,t}$ | Dirichlet concentration parameter for component $i$ |
| $\Delta w_{i,t}$ | Change in stock $i$ 's portfolio allocation |
| $V_t$ | Total portfolio value at the end of period $t$ |
| $C_t$ | Monetary transaction cost at time $t$ |
| $c_{\text{trans}}$ | Proportional transaction-cost rate |
| $R_{i,t}$ | Realized return of stock $i$ during period $t$ |
| $R_{net,t}$ | Net portfolio return after modeled trading costs |
| $r_t$ | RL reward at time $t$ |
| $A_t$ | Exponentially weighted first moment of net returns for DSR |

| Symbol | Meaning |
|---|---|
| $B_t$ | Exponentially weighted second moment of net returns for DSR |
| $D_t$ | Differential Sharpe Ratio reward |
| $\eta$ | DSR moving-average adaptation rate |
| $\varepsilon_{\mathrm{DSR}}$ | Numerical-stability constant in the DSR denominator |
| $\lambda_{\mathrm{turn}}$ | Turnover-regularization coefficient |
| $\gamma$ | PPO discount factor |
| $\lambda_{\mathrm{GAE}}$ | GAE bias-variance trade-off parameter |
| $\epsilon_{\mathrm{PPO}}$ | PPO clipping parameter |
| $\hat{A}_t$ | Estimated advantage used by PPO |
| $\delta_t$ | One-step temporal-difference error used by GAE |
| $\pi_\theta$ | PPO policy parameterized by $\theta$ |
| $V_\phi$ | PPO critic/value function parameterized by $\phi$ |
| $\rho_t$ | PPO probability ratio between new and old policies |
| $\mathcal{S}$ | Underlying environment state space |
| $\Omega$ | Observation space available to the agent |
| $\mathcal{A}$ | Action space |
| $\mathcal{T}$ | Environment transition dynamics |
| $\mathcal{R}$ | Reward function |
| $R_{net,t}$ | Realized net portfolio return used by M1/M2 and DSR |